

Kapittel 10

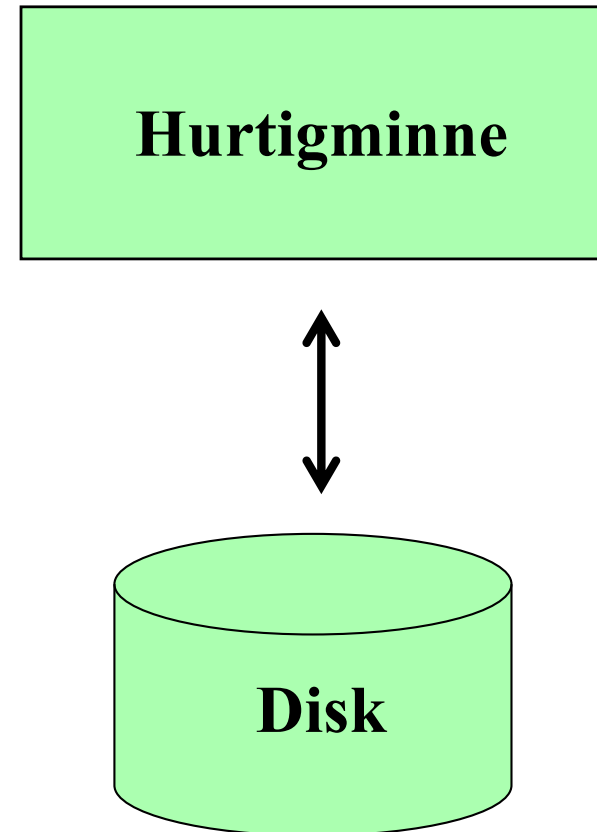
Transaksjoner

Læringsmål

- ☐ Forstå hva transaksjoner er, og hvilke egenskaper de bør ha.
- ☐ Kunne bekrefte og avbryte transaksjoner med SQL.
- ☐ Forstå hvilke utfordringer feilsituasjoner og samtidighet medfører for systemet med hensyn til transaksjoner.
- ☐ Forstå hvordan loggføring brukes for å håndtere feilsituasjoner.
- ☐ Forstå hvordan låser brukes for å håndtere samtidighetsproblemer.

Øyeblikksbilde av en database

- ❑ Data **lagres** på disk, men må leses inn i hurtigminnet for **beregning**.
- ❑ På et **bestemt tidspunkt** vil en del av databasen eksistere **kun i minnet**.
- ❑ Databasesystemer er i produksjon over lang tid.
- ❑ Må håndtere **feilsituasjoner**.



Hva er en «operasjon» ?

- ❑ SQL-setninger kan behandle **mange rader**:

UPDATE Ansatt

SET Lønn = Lønn * 1.1

- Lønn til samtlige ansatte blir oppdatert.
- 1 SQL-setning kan altså utføre mange «operasjoner».

- ❑ Motsatt kan vi ønske å betrakte **flere SQL-setninger** som én «sammensatt» operasjon.

- Eksempel fra Hobbyhuset: Ordrebestilling krever innsetting av rader i Ordre og Ordrelinje, samt oppdatering av Vare.

Transaksjonsbegrepet

- ❑ En **transaksjon** er en logisk operasjon mot databasen.
 - Kan involvere en eller flere tabeller.
 - Kan bestå av en eller flere SQL-setninger.

- ❑ Transaksjoner kan avsluttes på to måter:
 - **COMMIT**: bekreft, endringene gjøres permanente
 - **ROLLBACK**: angre, transaksjonen «rulles tilbake»

- ❑ Utfordringer:
 - **Feil** - for eksempel diskkrasj og strømbrudd
 - **Samtidige brukere** – som kan ødelegge for hverandre

Transaksjoner i SQL

❑ Kunde 5009 bestiller 5 enheter av produkt 44939 i en nettbutikk. Systemet oppretter ordre 20599 og flere tabeller må oppdateres:

BEGIN;

INSERT INTO Ordre(OrdreNr,KNr)

VALUES (20599,5009);

INSERT INTO Ordrelinje(OrdreNr,VNr,Antall)

VALUES (20599,'44939',5);

UPDATE Vare

SET Antall = Antall-5

WHERE VareNr='44939';

COMMIT;

❑ Etter den første/de to første innsettingene (INSERT) er ikke databasen i en lovlig tilstand.

Transaksjoner i PostgreSQL

❑ **Auto-commit** er standard. Kan skrus av:

```
SET autocommit = 0;
```

❑ Eller start **eksplisitt** med START TRANSACTION:

```
BEGIN TRANSACTION; ELLER BEGIN;
```

```
INSERT INTO Ordre(OrdreNr,KNr)
```

```
VALUES (20599,5009);
```

```
-- Flere kommandoer ...
```

```
COMMIT TRANSACTION; ELLER COMMIT;
```

ACID-egenskapene

Atomicity: Alle eller ingen av deloperasjonene i en transaksjon må fullføres.

Consistency: En transaksjon fører databasen fra en lovlig tilstand til en annen lovlig tilstand.

Isolation: Effekt av transaksjoner under utførelse skal ikke kunne observeres av andre transaksjoner.

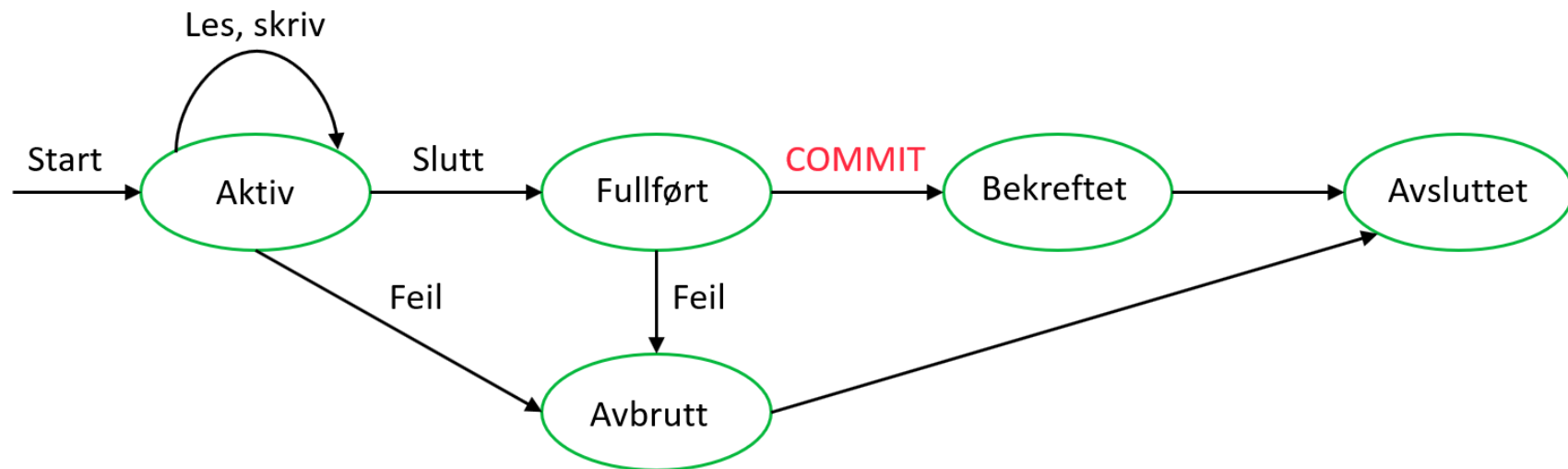
Durability: Effekt av fullførte (committed) transaksjoner lagres i databasen og skal ikke gå tapt på grunn av feil.

Transaksjonsloggen

TransID	Table	RowID	Attribute	Before	After
101	*** BEGINTRANS				
101	VARE	102375	ANTALL	112	113
101	VARE	102542	ANTALL	56	66
101	*** COMMIT				

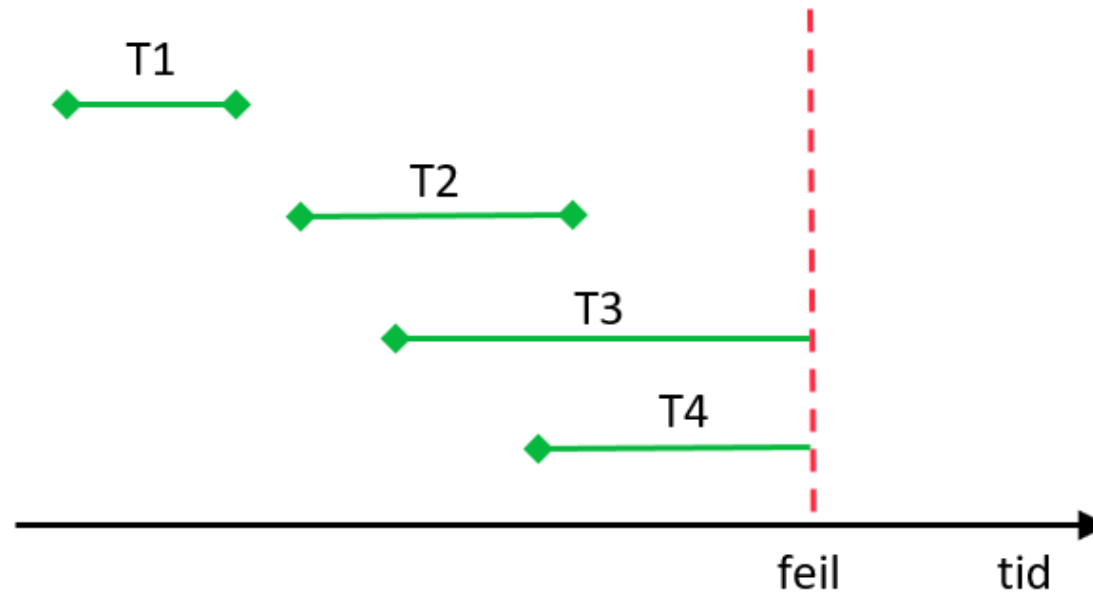
- ☐ Alt registreres i loggen før databasen blir oppdatert.
- ☐ Når **COMMIT** er skrevet til loggen, er beslutningen tatt.
- ☐ Loggen kan brukes for å oppdatere databasen.
- ☐ **ROLLBACK**: Eventuelle oppdateringer av databasen blir «rullet tilbake».
- ☐ Loggen brukes også ved gjenoppretting etter feil.

Livsløpet til en transaksjon



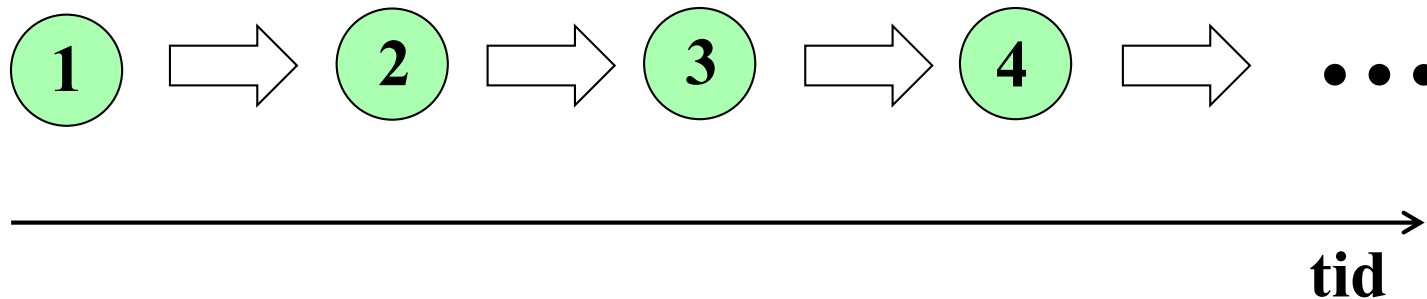
Avbrutte transaksjoner

- ❑ Fire transaksjoner i et hendelsesforløp der det oppstår en instansfeil.
- ❑ T1 og T2 har skrevet COMMIT til loggen og ble bekreftet før feilen inntraff.
- ❑ T3 og T4 var under utførelse. Kanskje hadde T3 rukket å gjennomføre flere skrive-operasjoner, mens T4 kun hadde gjennomført en leseoperasjon.



Transaksjoner og tilstander

- ❑ En **tilstand** er innholdet i databasen på et bestemt tidspunkt.
- ❑ En **transaksjon** bringer databasen fra én tilstand til en annen.
- ❑ Hvis vi tenker oss at kun én transaksjon blir utført av gangen, kan **livsløpet** til en database visualiseres slik:



Samtidighetskontroll

❑ Ønsker **flere samtidige brukere / transaksjoner**:

- Utnytte parallellitet (CPU og I/O)
- Redusere gjennomsnittlig ventetid (lange transaksjoner)

_____ 1 av gangen

_____ Flere samtidig

❑ Når er det trygt å tillate samtidig aksess?

- Mange brukere kan **lese** samme data samtidig
- To brukere kan skrive samtidig kun hvis de jobber med **forskjellige** deler av databasen.

Les-Beregn-Skriv

- ❑ Hva skjer når en «celle» på ytre lager skal oppdateres?

UPDATE Ansatt

SET Lønn = Lønn * 1.1

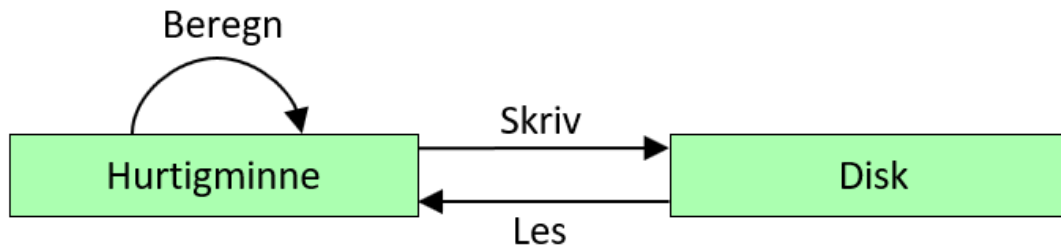
WHERE AnsNr = 14

- ❑ Transaksjonen består av følgende deloperasjoner:

1. **Les** post med AnsNr = 14 fra ytre lager
2. **Beregn** ny lønn
3. **Skriv** resultat til ytre lager

- ❑ Les-Beregn-Skriv er ikke en atomær operasjon.

- ❑ Det betyr at to oppdateringer kan «flettes» i tid !



Tapt oppdatering (lost update)

- ❑ Samtidighetsproblemer kan oppstå når to transaksjoner jobber med **samme data til samme tid**.
- ❑ Vi studerer hva som skjer når to transaksjoner skal oppdatere en verdi A på disken (en verdi i en bestemt kolonne i en bestemt rad i en tabell).

Lokal kopi T1	T1	Verdi A på disk	T2	Lokal kopi T2
120	Les inn	120		
110	Tell ned med 10	120	Les inn	120
110	Skriv til disk	110	Øk med 20	140
		140	Skriv til disk	140



- ❑ Oppdateringen til transaksjon T1 blir skrevet over av transaksjon T2 og går tapt. Hvis $A=120$ ved start, så får vi her $A=140$ ved slutt. Korrekt resultat er $A=130$ ($120+20-10$).

Angret oppdatering (dirty read)

- ❑ Transaksjon T2 bygger på et resultat fra transaksjon T1 som transaksjon T1 seinere «angrer» på at den produserte.

T1	A	T2
	120	
Tell ned A med 10	110	
	110	Les inn A og bruk denne verdien
ROLLBACK	120	



- ❑ Transaksjoner må ikke få lov til å avlese andres «mellomresultater».

Inkonsistent analyse (incorrect summary)

- ❑ Transaksjon T2 produserer en rapport basert på noen «gamle» og noen «nye» resultater.

	T1	A	B	T2	Lokal sum	
		10	50	Nullstill sum	0	
Den gamle verdien til A		10	50	Legg til A i sum	10	
Øk A med 10		20	50		10	
Øk B med 20		20	70		10	
Den nye verdien til B			70	Legg til B i sum	80	

tid ↓

- ❑ Selv 1 «leser» og 1 «skriver» kan altså gi samtidighetsproblemer !

Låser

- ❑ Generell løsning på samtidighetsproblemene:
Transaksjoner må vente på hverandre.
- ❑ En **lås** (lock) er en «ventemekanisme».
- ❑ En transaksjon kan låse større eller mindre deler:
 - hele databasen
 - en tabell
 - en rad i en tabell
 - en «celle» i en rad i en tabell
- ❑ Vi skiller mellom **skrivelåser** (exclusive locks) og **leselåser** (shared locks).

Løsning: Tapt oppdatering

Lokal kopi	T1	Verdi A på disk	T2	Lokal kopi
	Skrivelås på A	120		
120	Les inn	120	Skrivelås på A?	
110	Tell ned med 10	120	Vent	
110	Skriv til disk	110	Vent	
	Lås opp A	110	Vent	
		110	Les inn	110
		110	Øk med 20	130
		130	Skriv til disk	130
		130	Lås opp A	



- ❑ Angret oppdatering og inkonsistent analyse lar seg løse med låser på tilsvarende måter.

Serialiserbarhet

- ❑ Et **forløp** (schedule) er en «fletting» i tid av deloperasjonene til en samling transaksjoner.
 - Opplagt riktig: Kun 1 transaksjon av gangen = **sekvensiell forløp**.
 - **Men:** Det er mer effektivt med samtidige transaksjoner.
- ❑ Et **serialiserbart forløp** tillater samtidighet («fletting»), men har samme effekt som et sekvensielt forløp.

T1	A	B	T2
Skrivelås A	10	10	Skrivelås B
Tell ned A med 10	0	20	Øk B med 10 hvis større enn 0
Lås opp A	0	20	Lås opp B
Skrivelås B	0	20	Skrivelås A
Tell ned B med 10	0	10	Øk A med 10 hvis større enn 0
Lås opp B	0	10	Lås opp A



tid

NB! Ikke alle forløp er serialiserbare selv om de bruker låser.

Serialiserbarhet

- ❑ I serialiserbarhet er rekkefølgen på lese- og skriveoperasjonene viktig, da det kan oppstå konflikter her:

	T1	T2	Result
Same data item	Read Read Write Write	Read Write Read Write	No conflict Conflict Conflict Conflict
Different data item	Read/Write	Read/Write	No conflict

Øvelse

- Eksempel på sekvensiell plan1 (til venstre) og tilsvarende ikke-sekvensiell plan2 (til høyre) (A=100 \$, B=100 \$)
- Gir begge planene same resultat?
 - Er plan2 konflikt serialiserbar?

T_1	T_2
read(A) $A := A - 50$ write(A) read(B) $B := B + 50$ write(B)	read(A) $temp := A * 0.1$ $A := A - temp$ write(A) read(B) $B := B + temp$ write(B)

T_1	T_2
read(A) $A := A - 50$ write(A)	read(A) $temp := A * 0.1$ $A := A - temp$ write(A)
read(B) $B := B + 50$ write(B)	read(B) $B := B + temp$ write(B)

Øvelse

□ Eksempel på sekvensiell plan1 (til venstre) og tilsvarende ikke-sekvensiell plan2 (til høyre) (A=100 \$, B=100 \$)

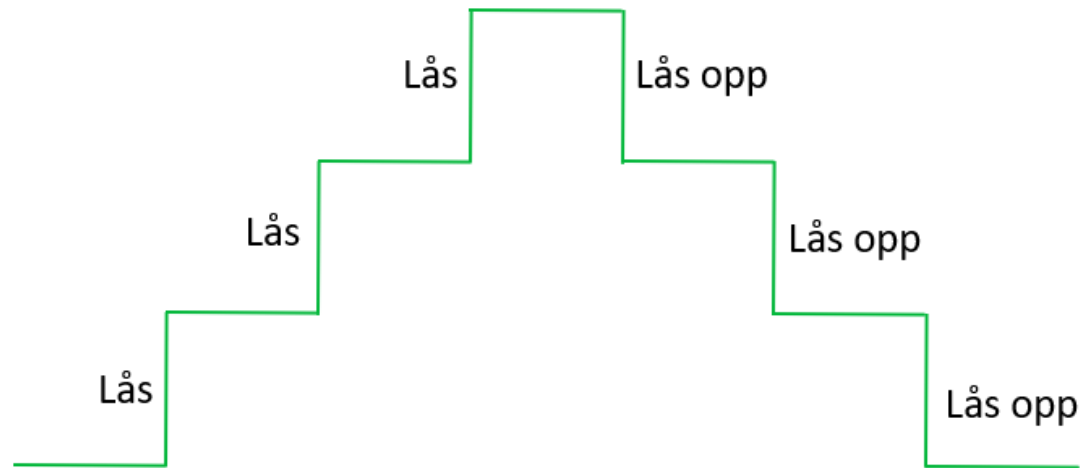
- Gir begge planene same resultat?
- Er plan2 konflikt serialiserbar?

T_1	T_2
read(A) $A := A - 50$ write(A) read(B) $B := B + 50$ write(B)	read(A) $temp := A * 0.1$ $A := A - temp$ write(A) read(B) $B := B + temp$ write(B)

T_1	T_2
read(A) $A := A - 50$ write(A) read(B) $B := B + 50$ write(B)	read(A) $temp := A * 0.1$ $A := A - temp$ write(A) read(B) $B := B + temp$ write(B)

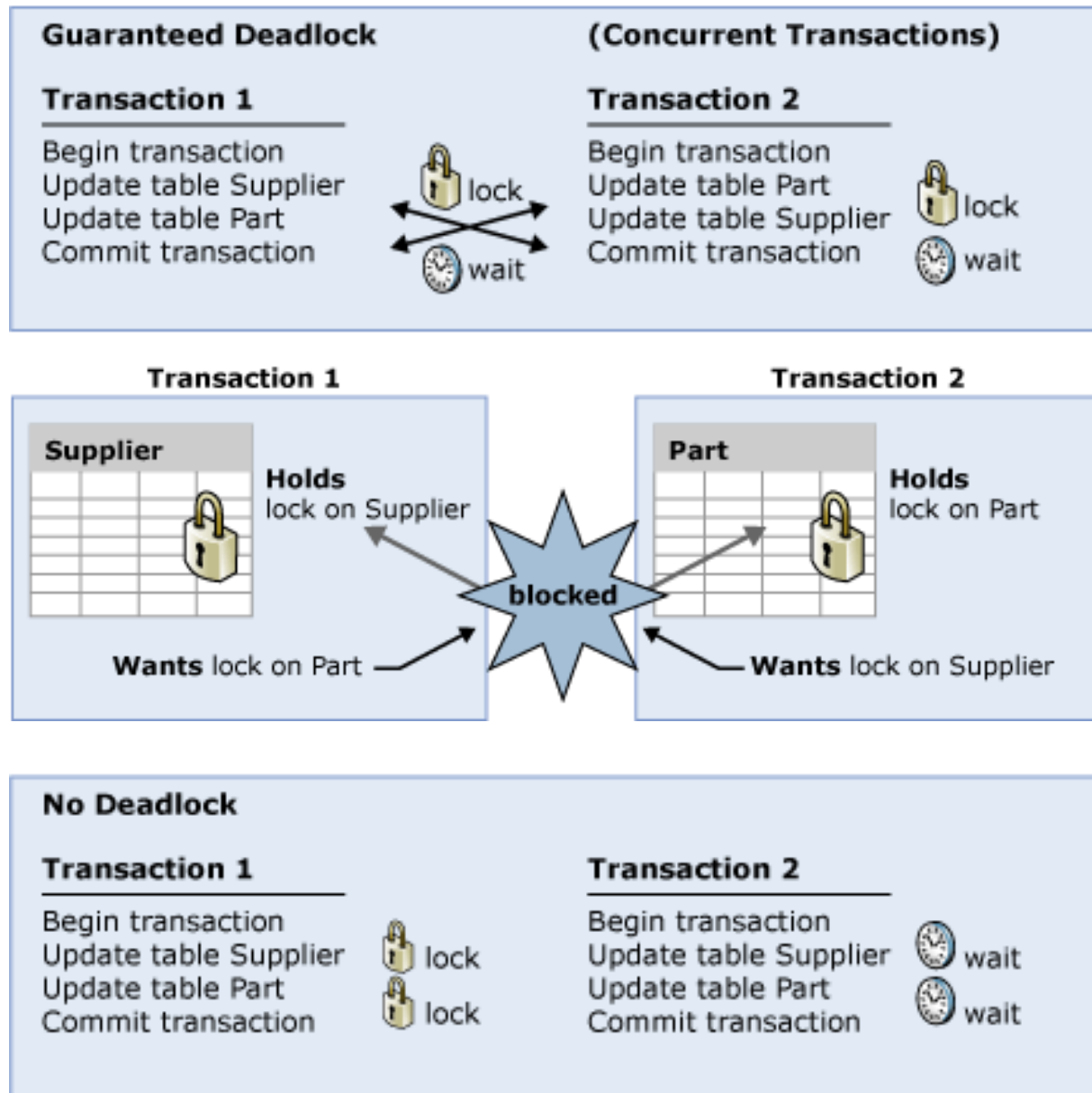
Tofaselåsing

- ❑ En transaksjon følger reglene for **tofaselåsing** hvis alle låseoperasjoner gjøres før første frigivelse (opplåsing).
 - Det vil si, ingen låsing etter første opplåsing!



- ❑ **Følgende gjelder:** Hvis alle transaksjoner følger tofaselåsing vil ethvert forløp bli serialiserbart.

Vranglås



Håndtere vranglås

❑ Oppdage og «løse opp» vranglås

- Bygge opp en **ventegraf**: Hvis transaksjon T1 må vente på transaksjon T2, så legg til en kant (pil) fra T1 til T2.
- Av og til: Gå gjennom ventegrafen og sjekk om det har oppstått en **sykel**, for eksempel at T1 venter på T2 som venter på T3 som igjen venter på T1.
- Velg en av transaksjonene i cykelen. **Avbryt** transaksjonen («rull den tilbake»). Gjennomfør de andre, og start avbrutt transaksjon etterpå.

❑ Forhindre vranglås

- Alle transaksjoner gis et unikt **tidsstempel**.
- Hvis en transaksjon vil måtte vente på en **eldre** transaksjon blir den **avbrutt**. Det betyr at yngre transaksjoner aldri vil vente på eldre og dermed kan det ikke oppstå sykler.

Isolasjonsnivåer

- ❑ Godta litt «innblanding» av effektivitetshensyn?
- ❑ Anomalier:
 - Usikker lesing: T1 kan lese data skrevet av T2 før T2 har skrevet COMMIT.
 - Ikke-repeterbar lesing: T1 kan få to forskjellige svar fordi T2 endrer og bekrefter.
 - Fantomer: T1 oppdager nye rader satt inn av T2.

Isolasjonsnivå	Fantomer	Ikke-repeterbar lesing	Usikker lesing
SERIALIZABLE	Nei	Nei	Nei
REPEATABLE READ	Ja	Nei	Nei
READ COMMITTED	Ja	Ja	Nei
READ UNCOMMITTED	Ja	Ja	Ja

Kapittel 11

Databaseadministrasjon

Sikkerhetskopiering og gjenoppbygging

- ❑ Typer av **sikkerhetskopiering**

- Full/inkrementell

- ❑ Sikkerhetskopiering til skyen

- ❑ Verktøy for sikkerhetskopiering og **gjenoppbygging** (recovery)

- ❑ **Transaksjoner**

- Oppdateringer skrives først til **transaksjonsloggen** så til databasen.

- Siste sikkerhetskopi + transaksjonsloggen brukes ved gjenoppbygging for å bringe databasen tilbake i en **konsistent** tilstand.

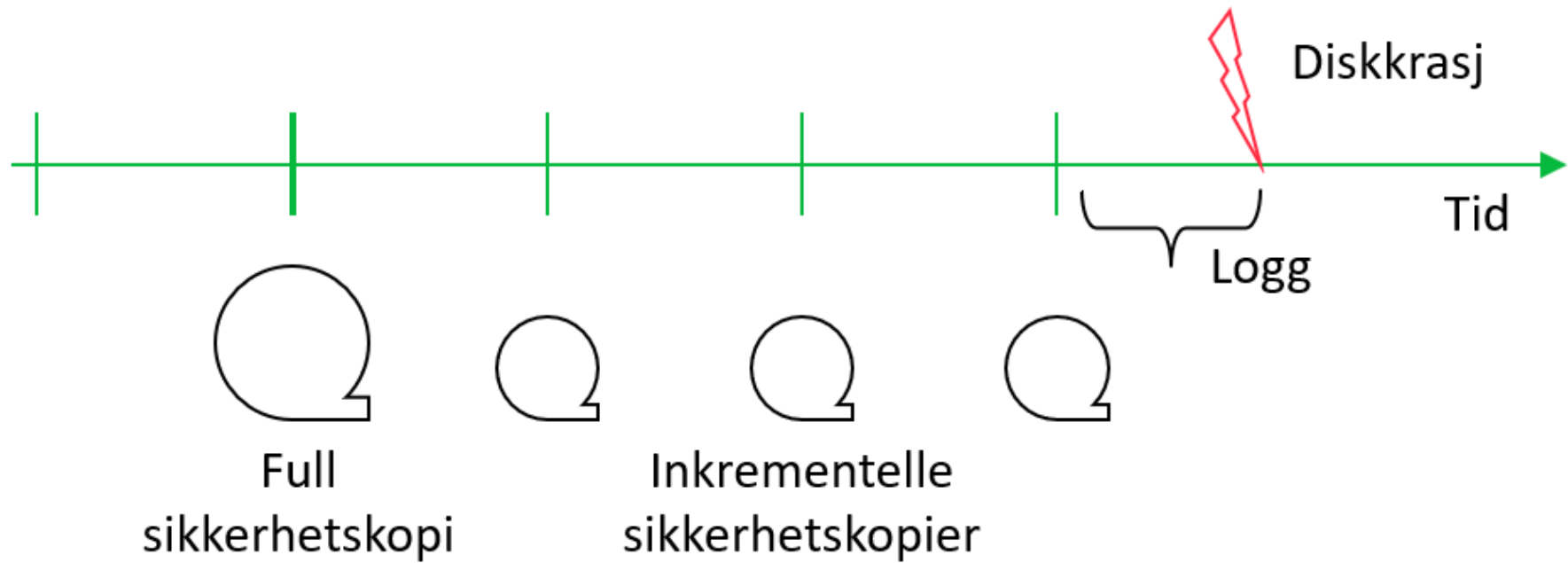
- ❑ **Rutiner for sikkerhetskopiering**

- Tidspunkter

- Oppbevaring av sikkerhetskopier

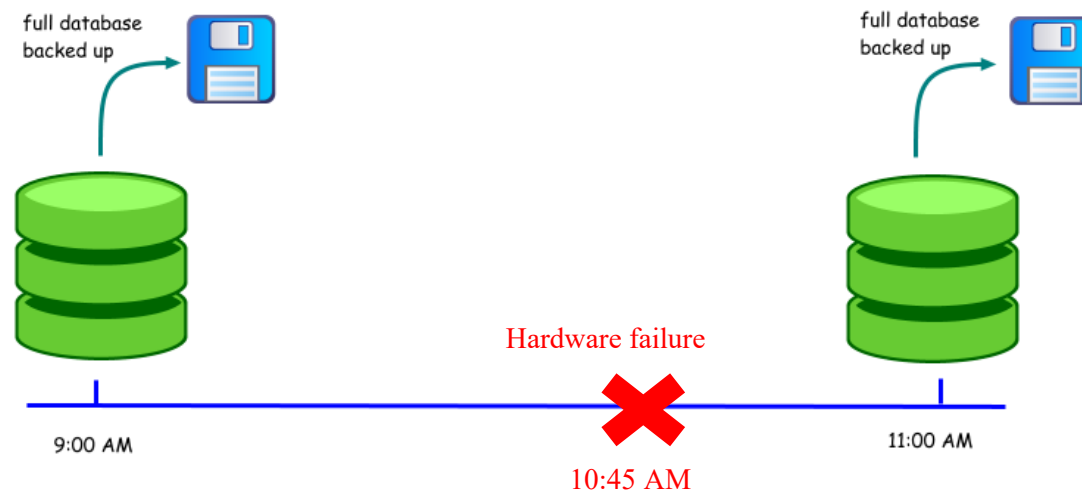


Sikkerhetskopier og gjenoppbygging



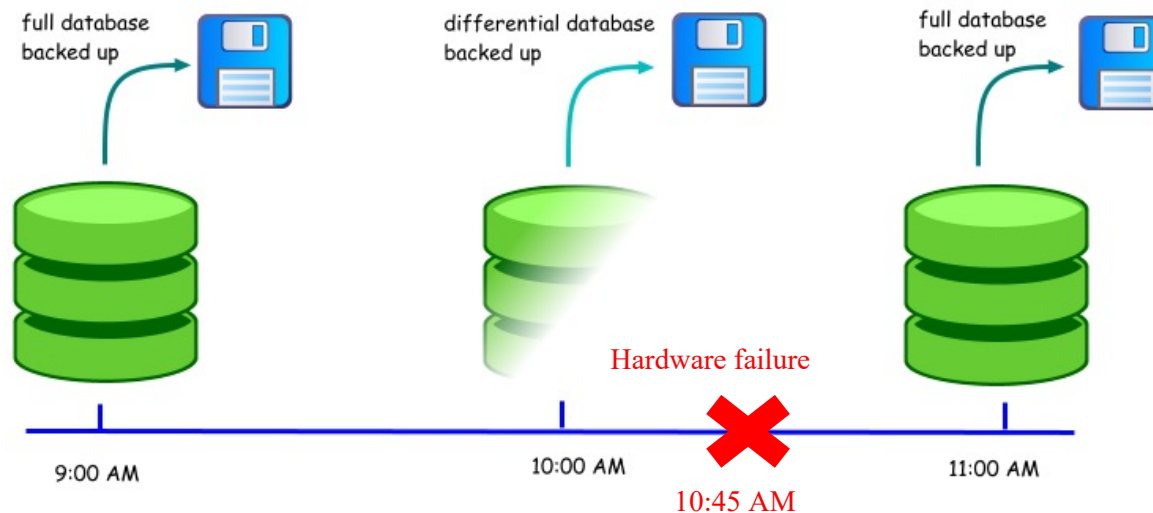
Eksempel

- ❑ Anta at det var en maskinvarefeil klokken 10:45. Hvis du bare brukte sikkerhetskopier, ville du ha mistet 105 minutter med arbeid.
- ❑ Det siste tidspunktet du kan gjenopprette databasen til er klokken 09:00, siden det var da den siste fullstendige sikkerhetskopien ble tatt.



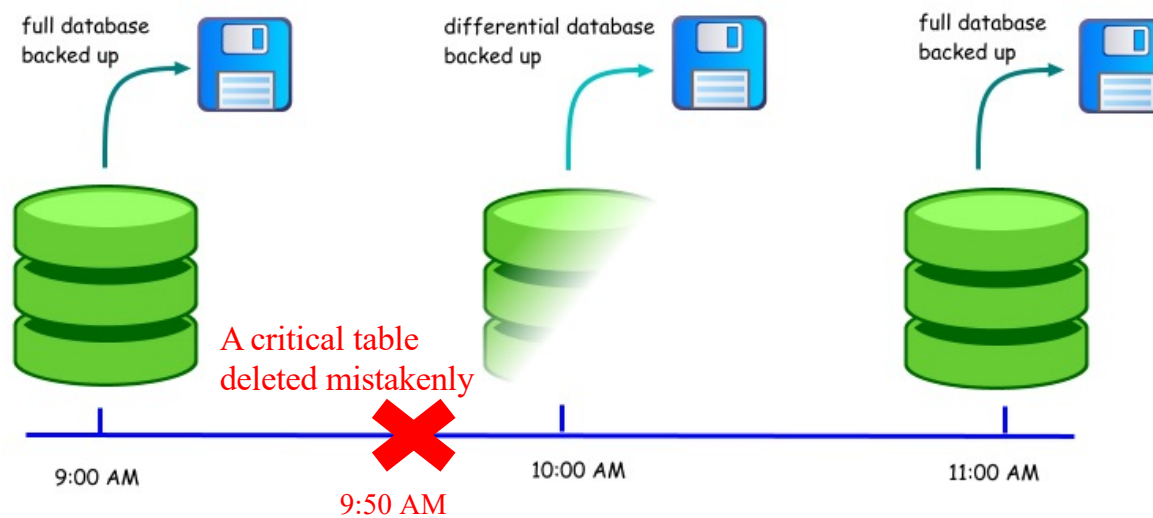
Øvelse

- ❑ Differensiell sikkerhetskopiering er en inkrementell sikkerhetskopiering av alle endringer som er gjort siden forrige fullstendige sikkerhetskopiering.
- ❑ Hva kan vi gjøre for å gjenopprette databasen til tilstanden før feilen?



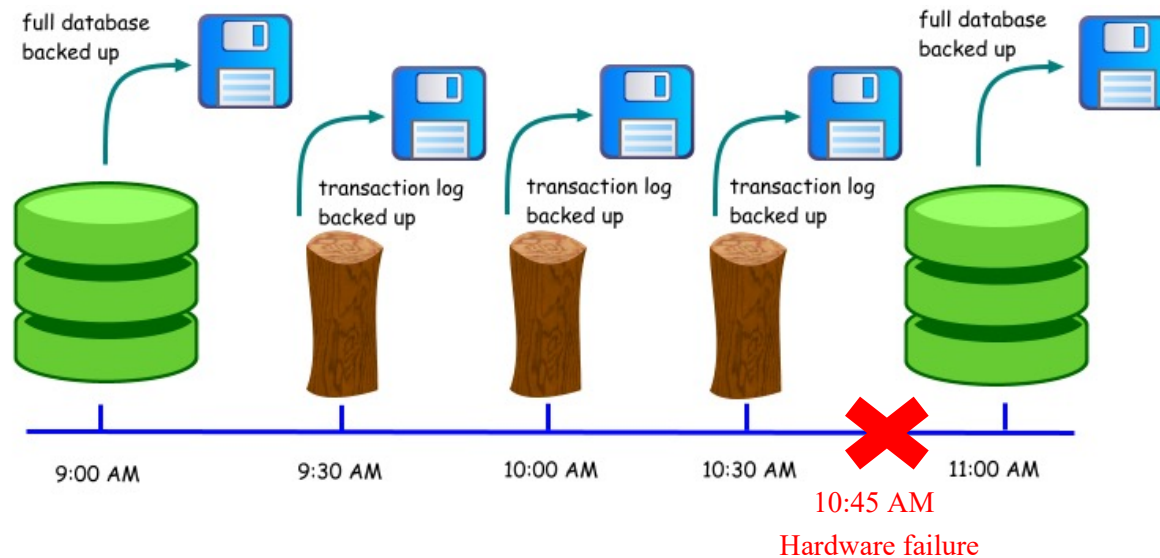
Øvelse

- ❑ Anta at en bruker slettet en kritisk tabell klokken 09:50. Kan du gjenopprette til tidspunktet rett før slettingen?



Øvelse

- ❑ Anta nå at transaksjonsloggen sikkerhetskopieres hvert 30. minutt.
- ❑ Hvis det oppstår en maskinvarefeil klokken 10:45, hvordan bringer vi databasen til korrekt tilstand etter feilen?



Gjenoppretting

❑ DBMS bør tilby følgende fasiliteter for å hjelpe med gjenoppretting:

- Sikkerhetskopieringsmekanisme, som lager periodiske sikkerhetskopier av databasen.
- Loggingsfasiliteter, som holder oversikt over gjeldende status for transaksjoner og databaseendringer.

- Loggfiler inneholder informasjon om alle oppdateringer til databasen (informasjon om transaksjoner og checkpoints).

- **Hvorfor Checkpoints?**

- Når det oppstår en feil, vet vi kanskje ikke hvor langt vi må søke i loggfilen, og vi kan ende opp med å gjøre om transaksjoner som er trygt skrevet til disken.

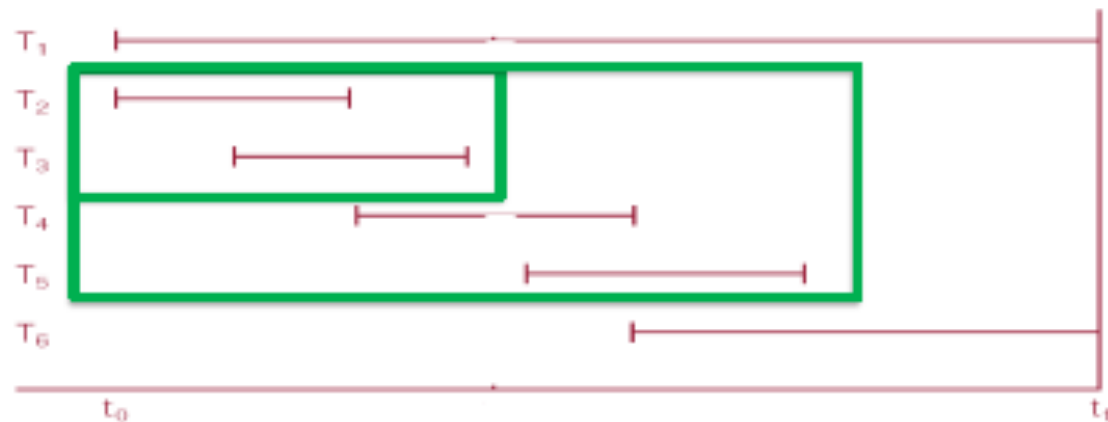
- Dermed vil checkpoints begrense antall søk i loggfilen og omgjøringen av transaksjoner som allerede er lagret på disken.

- Når det oppstår en feil, utfør alle transaksjoner som er utført siden checkpoint på nytt (redo), og gjør undo av alle transaksjoner som var aktive ved krasjtidspunktet.

Gjenoppretting

- ❑ Alle transaksjoner som er **committed** og dataoppdateringene deres skrives til disk før checkpoint, blir ikke gjort på nytt.
- ❑ Vi gjør bare om transaksjoner som var aktive ved checkpoint og eventuelle påfølgende transaksjoner der «start»- og «commit» informasjon lagres i loggfilen.
- ❑ Hvis en transaksjon er aktiv på feiltidspunktet, må den gjøres rolles tilbake hvis den startet før det siste kontrollpunktet.

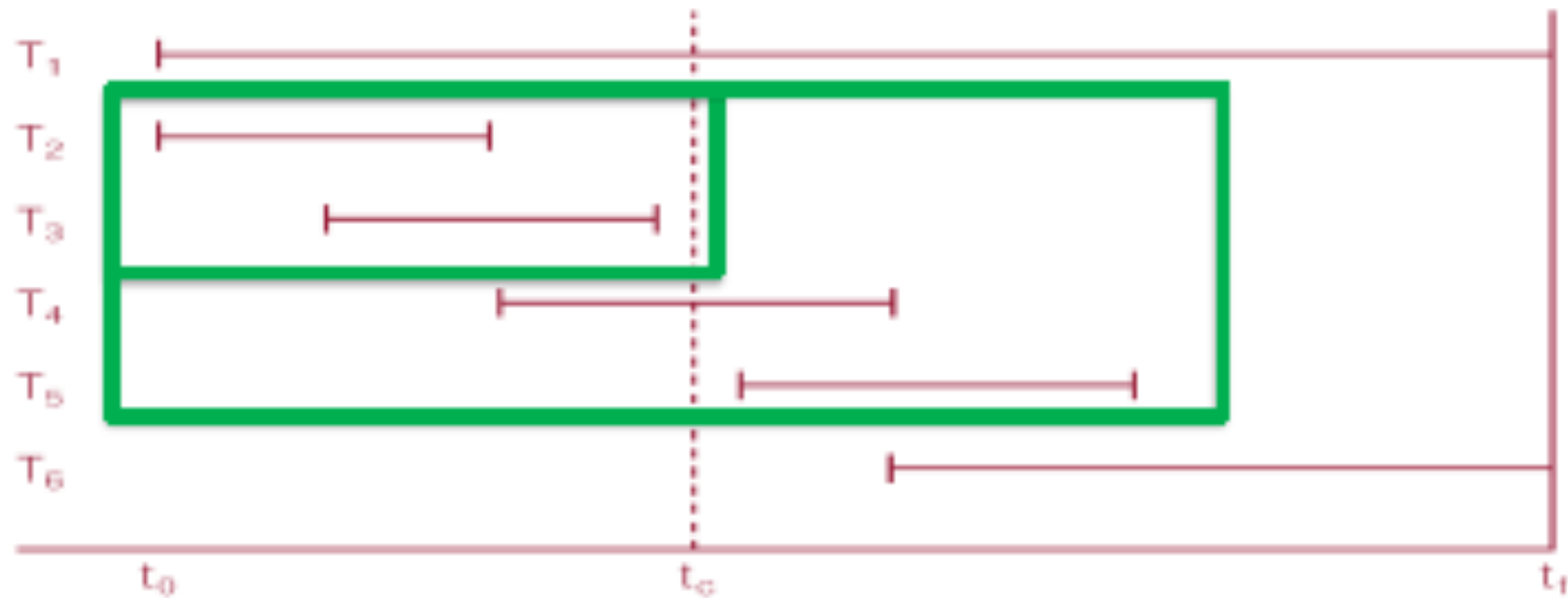
Øvelse: Hvilke transaksjoner skal omgjøres og hvilke skal rolles tilbake?



Øvelse

The DBMS starts at time t_0 and fails at time t_f . Transactions T_2 , T_3 , T_4 , and T_5 have committed before the time of failure. The data for T_2 and T_3 has been written to disk before the failure, but no information stored about that in the log file. T_1 , T_6 have not committed at the point of the crash.

Which transactions will be undone and which ones will be redone?



Øvelse

- Hvilke transaksjoner skal omgjøres og hvilke skal rolles tilbake?

